進捗確認API仕様書

作成日: 2026-01-21 16:45
最終更新: 2026-03-12 18:28
バージョン: 2.0.0
対象画面: 実験管理者 - 進捗確認画面、実験作業者 - 実験記録画面
総API数: 6個
実装状況: 6個実装済み、0個未実装

📊 実装状況サマリー

カテゴリ	実装済み	未実装	合計
進捗確認	6	0	6
合計	**6**	**0**	**6**

完了率: 100.0% (6/6)

🎯 API一覧

#	API名	メソッド	エンドポイント	状態	優先度
1	タスク詳細取得	GET	`/api/v1/progress/projects/:projectName/methods/:methodId/tasks/:taskNo/details`	🟢 実装済み	⭐⭐ 高
2	進捗データ取得	GET	`/api/v1/progress/projects/:projectName/methods/:methodId/tasks/:taskNo/transitions`	🟢 実装済み	⭐⭐ 高
3	タスク一覧取得（実験作業者用）	GET	`/api/v1/progress/projects/:projectName/worker-tasks`	🟢 実装済み	⭐⭐ 高
4	タスクステータス更新	PATCH	`/api/v1/progress/projects/:projectName/methods/:methodId/tasks/:taskNo/status`	🟢 実装済み	⭐⭐ 高
5	トランジションステータス更新	PATCH	`/api/v1/progress/projects/:projectName/methods/:methodId/transitions/:transitionId/status`	🟢 実装済み	⭐⭐ 高
6	作業詳細一括取得	GET	`/api/v1/progress/projects/:projectName/work-details`	🟢 実装済み	⭐⭐ 高

📋 概要

進捗確認APIは、プロジェクト内のタスク進捗状況を確認するための機能を提供します。

主な特徴

1. **メソッド別進捗確認**: メソッド単位でタスクの進捗を集約
2. **タスク詳細情報**: 各タスクの詳細情報（担当者、ステータス、実験記録）を取得

3. **リアルタイム進捗**: 最新のタスクステータスを反映

🔐 共通仕様

認証・認可

認証方式: JWT + Session Hybrid

必須ヘッダー:

```
Authorization: Bearer maiml_session-{sessionId}_{timestamp}
```

organization_id の取得:

- ログイン時に `X-Organization-Id` ヘッダーから組織IDを取得し、セッションに保存します
- ログイン後のAPIでは、 `Authorization` ヘッダーのセッショントークンから自動的に取得されます
- リクエストヘッダーで `X-Organization-Id` を指定する必要はありません
- `authMiddleware` がセッション情報を検証し、ユーザー情報（organization_idを含む）を `req.user` に格納します

権限要件:

- すべてのAPI: `experiment_admin` または `system_admin` ロール必須

ミドルウェアチェーン:

```
router.METHOD('/path',
  authMiddleware,           // 1. JWT認証（セッション検証、req.userに格納）
  permissionCheck([...]),   // 2. 権限チェック
  controller.handler        // 3. コントローラー処理
);
```

ミドルウェアの動作:

1. `authMiddleware` : JWTトークン（ `maiml_session-{sessionId}_{timestamp}` ）を検証し、セッション情報を取得して `req.user` に格納（organizationIdを含む）
2. `permissionCheck` : ユーザーのロール（ `req.user.role` ）を検証
3. コントローラー: `req.user.organizationId` を使用してテナント分離されたデータにアクセス

マルチテナント分離

- すべてのDBクエリに `organization_id` フィルタ必須

統一レスポンス形式

成功レスポンス:

```
{
  "success": true,
  "data": { ... },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラーレスポンス:

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "エラーメッセージ",
    "details": []
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

📋 API詳細仕様

1. タスク詳細取得 🟢

エンドポイント: `GET /api/v1/progress/projects/:projectName/methods/:methodId/tasks/:taskNo/details`

説明: プロジェクト名・メソッドID・タスク連番でタスクを特定し、タスク基本情報・トランジション進捗一覧・統計情報を取得する

認証: 必須

権限: `experiment_admin`（機能CD: `040000`、読取専用）

優先度: ⭐⭐⭐ 高

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`projectName`	string	○	プロジェクト名
`methodId`	string	○	メソッドID（MAIMLファイル内のmethod id）

`taskNo`	integer	○	タスク連番（1以上の整数）

レスポンス

成功時 (200 OK):


```
{
  "success": true,
  "data": {
    "taskId": "550e8400-e29b-41d4-a716-446655440000__method1__3",
    "projectId": "550e8400-e29b-41d4-a716-446655440000",
    "projectName": "Sample Project",
    "methodId": "method1",
    "experimentOrder": 1,
    "taskNo": 3,
    "taskStatus": "in_progress",
    "taskStatusNum": 1,
    "experimentUserId": "880e8400-e29b-41d4-a716-446655440003",
    "experimentUserName": "Worker Name",
    "experimentStartAt": "2026-01-21T10:30:00.000Z",
    "experimentEndAt": null,
    "reviewedAt": null,
    "reviewUserId": null,
    "transitionProgress": [
      {
        "transitionId": "t1",
        "status": 5,
        "progressRate": 100,
        "updatedAt": "2026-01-21T14:00:00.000Z"
      },
      {
        "transitionId": "t2",
        "status": 3,
        "progressRate": 60,
        "updatedAt": "2026-01-21T15:30:00.000Z"
      },
      {
        "transitionId": "t3",
        "status": 0,
        "progressRate": null,
        "updatedAt": null
      }
    ],
    "stats": {
      "totalTransitions": 3,
      "completed": 1,
      "inProgress": 1,
      "notStarted": 1,
      "progressPercentage": 53.3
    },
    "createdAt": "2026-01-21T10:00:00.000Z",
    "updatedAt": "2026-01-21T15:30:00.000Z"
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```


レスポンス data フィールド詳細:

フィールド	型	説明
`taskId`	string	タスクID文字列（`{projectId}__{methodId}__{taskNo}` 形式）
`projectId`	string (UUID)	プロジェクトID
`projectName`	string	プロジェクト名
`methodId`	string	メソッドID
`experimentOrder`	number	実験順序（`project_task.experiment_order`）
`taskNo`	number	タスク連番（`project_task_detail.task_no`）
`taskStatus`	string	タスクステータス文字列（`pending` / `in_progress` / `review_waiting` / `completed` / `failed`）
`taskStatusNum`	number | null	タスクステータス数値（DB の `task_status` 値）
`experimentUserId`	string | null	担当作業者ID
`experimentUserName`	string | null	担当作業者名
`experimentStartAt`	string | null	実験開始日時（ISO 8601）
`experimentEndAt`	string | null	実験終了日時（ISO 8601）
`reviewedAt`	string | null	レビュー完了日時（ISO 8601）
`reviewUserId`	string | null	レビュアーID
`transitionProgress`	array	トランジション進捗一覧（後述）
`stats`	object	進捗統計情報（後述）
`createdAt`	string | null	レコード作成日時（ISO 8601）
`updatedAt`	string | null	レコード更新日時（ISO 8601）

transitionProgress[] フィールド詳細:

フィールド	型	説明
`transitionId`	string	トランジションID
`status`	number | null	ステータス数値（0:初期, 1:準備中, 2:準備完了, 3:実行中, 4:一時停止, 5:完了, 6:エラー）
`progressRate`	number | null	進捗率（0〜100）
`updatedAt`	string | null	最終更新日時（ISO 8601）

stats フィールド詳細:

フィールド	型	説明
`totalTransitions`	number	トランジション総数
`completed`	number	完了数（status = 5）
`inProgress`	number	作業中数（status = 1〜4）
`notStarted`	number	未着手数（status = 0 または NULL）
`progressPercentage`	number	進捗率（avg_progress_rate から算出、小数点1桁）

エラー時:

- 400: バリデーションエラー（taskNo が1未満など）
- 401: 認証エラー
- 403: 権限不足
- 404: プロジェクトまたはタスクが存在しない（ `RESOURCE_NOT_FOUND` ）
- 500: システムエラー

実装要件

ミドルウェアチェーン:

```
router.get(
  '/projects/:projectName/methods/:methodId/tasks/:taskNo/details',
  authMiddleware,                    // 1. JWT認証
  checkPermission('040000', 0),      // 2. 権限チェック（実験管理者メニュー、読取専用）
  [
    validateParamString('projectName', 1, 255),
    validateParamString('methodId', 1, 255),
    validateParamInt('taskNo', 1),
  ],
  handleValidationErrors,            // 3. バリデーションエラーハンドリング
  projectController.getTaskDetail    // 4. コントローラー
);
```

コントローラー処理フロー:

```
// 1. projectName → projectId 解決
const projectResult = await pool.query(
  'SELECT project_id FROM project WHERE organization_id = $1 AND project_name = $2',
  [organizationId, projectName]
);
// → 404 if not found

// 2. taskId 文字列生成（フロントエンド識別子用）
const taskId = `${projectId}__${methodId}__${taskNo}`;

// 3. Repository 呼び出し
const taskDetail = await projectTaskDetailRepository.findTaskDetail(
  organizationId, projectId, methodId, taskNo, taskId
);
// → 404 if not found
```

使用 SQL ファイル:

SQLファイル	説明
`backend/src/sql/queries/tasks/findDetailByTaskId.sql`	タスク基本情報 + transition_data 集計（ `$1` : projectId, `$2` : methodId, `$3` : taskNo, `$4` : organizationId）
`backend/src/sql/queries/tasks/findTransitionsByTaskId.sql`	トランジション進捗個別レコード一覧（同パラメータ）

`findDetailByTaskId.sql` 取得カラム:

- `project_id`, `project_name`（`project` テーブル）
- `method_id`, `experiment_order`, `task_name`（`project_task` テーブル）
- `task_no`, `task_status`, `experiment_user_id`, `experiment_start_at`, `experiment_end_at`, `reviewed_at`, `review_user_id`, `create_at`, `update_at`（`project_task_detail` テーブル）
- `user_name AS experiment_user_name`（`organization_user` LEFT JOIN）
- `total_transitions`, `completed_transitions`, `inprogress_transitions`, `notstarted_transitions`, `avg_progress_rate`（`transition_data` LEFT JOIN + 集計）

`findTransitionsByTaskId.sql` 取得カラム:

- `transition_id`, `status`, `progress_rate`, `create_at`, `update_at`（`transition_data` テーブル）

タスクステータス数値 → 文字列 変換マップ:

```
const TASK_STATUS_MAP: Record<number, string> = {
  0: 'pending',
  1: 'in_progress',
  2: 'failed',
  3: 'review_waiting',
  4: 'completed',
};
```

実装状況

実装済み箇所:

- ✅ ルート定義: `backend/src/routes/progressRoutes.ts`
  - `checkPermission('040000', 0)` による権限チェック実装済み
  - 機能CD: `040000`（実験管理者メニュー）、権限レベル: `0`（読取専用）
- ✅ バリデーション: `validateParamString` / `validateParamInt` による検証実装済み
- ✅ コントローラー: `backend/src/controllers/projectController.ts` - `getTaskDetail()`
- ✅ Repository: `backend/src/repositories/projectTaskDetailRepository.ts` - `findTaskDetail()`
- ✅ SQL: `backend/src/sql/queries/tasks/findDetailByTaskId.sql`
- ✅ SQL: `backend/src/sql/queries/tasks/findTransitionsByTaskId.sql`

🔒 セキュリティ考慮事項

マルチテナント分離

必須チェック:

- ☐ すべてのDBクエリに `organization_id` フィルタ
- ☐ クロステナントアクセスの防止
- ☐ タスク担当者の所属組織確認

実装例:

```
// ✅ 良い例
const task = await db
  .selectFrom('project_task')
  .where('task_id', '=', taskId)
  .where('organization_id', '=', organizationId)  // 必須
  .selectAll()
  .executeTakeFirst();

// ❌ 悪い例
const task = await db
  .selectFrom('project_task')
  .where('task_id', '=', taskId)  // organization_idフィルタなし
  .selectAll()
  .executeTakeFirst();
```

進捗情報の機密性

考慮事項:

- タスク詳細は担当者と管理者のみ閲覧可能
- 実験記録データは機密情報として扱う
- 問題報告は組織内のみ共有

🔧 実装ガイドライン

タスクステータスの定義

```
enum TaskStatus {
  PENDING = 'pending',         // 未着手
  IN_PROGRESS = 'in_progress', // 実行中
  COMPLETED = 'completed',     // 完了
  FAILED = 'failed',           // 失敗
  CANCELLED = 'cancelled'      // キャンセル
}
```

進捗計算ロジック

```
const calculateTaskProgress = (taskDetails: TaskDetail[]) => {
  const totalSteps = taskDetails.length;
  const completedSteps = taskDetails.filter(
```

```
    detail => detail.status === 'completed'
  ).length;

  return {
    total_steps: totalSteps,
    completed_steps: completedSteps,
    progress_percentage: totalSteps > 0
      ? Math.round((completedSteps / totalSteps) * 100 * 10) / 10
      : 0
  };
};
```

キャッシュ戦略

進捗情報のキャッシュ:

- メソッド別タスク一覧: 5分間キャッシュ
- タスク詳細: キャッシュなし（常に最新情報を取得）

```
import NodeCache from 'node-cache';

const progressCache = new NodeCache({ stdTTL: 300 }); // 5分

const getCachedMethodTasks = async (projectId: string, methodId: string) => {
  const cacheKey = `method_tasks_${projectId}_${methodId}`;

  let tasks = progressCache.get(cacheKey);
  if (!tasks) {
    tasks = await fetchMethodTasks(projectId, methodId);
    progressCache.set(cacheKey, tasks);
  }

  return tasks;
};
```

---

3. 進捗データ取得API 🟢

エンドポイント: `GET /api/v1/progress/projects/:projectName/methods/:methodId/tasks/:taskNo/transitions`

説明: 指定タスクに紐づくトランジションの進捗データを取得

認証: 必須

権限: `experiment_worker`（機能CD: `050000`、読取専用）

優先度: ⭐⭐⭐ 高

実装状況: ✅ 実装済み

- コントローラー: `backend/src/controllers/progressController.ts` - `getProgressData()`
- サービス: `backend/src/services/progressService.ts` - `ProgressService.getProgressData()`

- リポジトリ: `backend/src/repositories/transitionDataRepository.ts` - `getLatestRecordByTransition()`
- ルート定義: `backend/src/routes/progressRoutes.ts` - `checkPermission('050000', 0)`

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`projectName`	string	○	プロジェクト名
`methodId`	string	○	メソッドID（MAIMLファイル内のmethod id）
`taskNo`	Integer	○	タスク連番

レスポンス

成功時 (200 OK):


```
{
  "success": true,
  "data": {
    "project_id": "550e8400-e29b-41d4-a716-446655440000",
    "project_name": "Sample Project",
    "method_id": "method1",
    "method_name": "method1",
    "task_no": 1,
    "transitions": [
      {
        "transition_id": "t1",
        "transition_name": "材料準備",
        "status": 5,
        "progress": 100,
        "last_updated": "2026-01-21T14:00:00.000Z",
        "material_count": 3,
        "material_actual_count": 3,
        "condition_count": 2,
        "condition_actual_count": 2,
        "result_count": 1,
        "result_actual_count": 1
      },
      {
        "transition_id": "t2",
        "transition_name": "反応実験",
        "status": 3,
        "progress": 45,
        "last_updated": "2026-01-21T15:30:00.000Z",
        "material_count": 4,
        "material_actual_count": 2,
        "condition_count": 3,
        "condition_actual_count": 1,
        "result_count": 2,
        "result_actual_count": 0
      },
      {
        "transition_id": "t3",
        "transition_name": "結果測定",
        "status": 0,
```

```
        "progress": 0,
        "last_updated": null,
        "material_count": 2,
        "material_actual_count": 0,
        "condition_count": 1,
        "condition_actual_count": 0,
        "result_count": 3,
        "result_actual_count": 0
      }
    ],
    "overall_progress": {
      "total_transitions": 3,
      "completed": 1,
      "in_progress": 1,
      "not_started": 1,
      "overall_percentage": 48.3
    }
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```


レスポンス data フィールド詳細:

フィールド	型	説明
`project_id`	string (UUID)	プロジェクトID
`project_name`	string	プロジェクト名
`method_id`	string	メソッドID
`method_name`	string	メソッド名（現時点では method_id の値をそのまま返却）
`task_no`	number	タスク連番（整数）
`transitions`	array	トランジション進捗一覧（後述）
`overall_progress`	object	全体進捗情報（後述）

transitions[] フィールド詳細:

フィールド	型	説明
`transition_id`	string	トランジションID
`transition_name`	string	トランジション名（MAIMLラベル、未設定時はID）
`status`	number	トランジションステータス数値（0〜6、後述）
`progress`	number	進捗率（0〜100）。status=5(COMPLETED)時は自動的に100、status=0(INITIAL)時は0に補正される
`last_updated`	string | null	最終更新日時（ISO 8601）
`material_count`	number	MAIMLに定義された材料テンプレート数
`material_actual_count`	number	DBに登録された材料実績数
`condition_count`	number	MAIMLに定義された条件テンプレート数
`condition_actual_count`	number	DBに登録された条件実績数
`result_count`	number	MAIMLに定義された結果テンプレート数

`result_actual_count`	number	DBに登録された結果実績数

トランジションステータス数値定義:

数値	名称	説明
`0`	INITIAL	初期（未着手）
`1`	PREPARING	準備中
`2`	READY	準備完了
`3`	IN_PROGRESS	実行中
`4`	PAUSED	一時停止
`5`	COMPLETED	完了
`6`	ERROR	エラー

overall_progress フィールド詳細:

フィールド	型	説明
`total_transitions`	number	トランジション総数
`completed`	number	完了数（status = 5）
`in_progress`	number	作業中数（PREPARING(1) / READY(2) / IN_PROGRESS(3) / PAUSED(4) / ERROR(6)）
`not_started`	number	未着手数（status = 0）
`overall_percentage`	number	全体進捗率（全トランジションの progress 平均値、小数点1桁）

エラー時:

- 400: バリデーションエラー
- 401: 認証エラー
- 403: 権限不足
- 404: プロジェクト（ `RESOURCE_NOT_FOUND` ）、メソッド（ `METHOD_NOT_FOUND` ）、またはタスク（ `TASK_NOT_FOUND` ）が存在しない
- 500: MAIMLダウンロード失敗（ `MAIML_DOWNLOAD_FAILED` ） / MAIMLパース失敗（ `MAIML_PARSE_FAILED` ） / DBエラー（ `DATABASE_ERROR` ）

実装要件

コントローラー処理フロー:

```
// 1. projectName → projectId 解決（projects/findByName.sql）
const project = await pgp.oneOrNone(projectQuery, [projectName, organizationId]);
// → 404 RESOURCE_NOT_FOUND if not found

// 2. ProgressService 呼び出し
const result = await progressService.getProgressData(projectId, methodId, taskNo, organizationId);
```

ProgressService 処理フロー:

1. **タスク基本情報取得**: `projectTaskRepository.getTaskByProjectMethodTaskNo(projectId, methodId, taskNo, organizationId)` → タスクなし時 `TASK_NOT_FOUND`
2. **MAIMLダウンロード＆パース**: `storageService.downloadFile(organizationId, 'projects/{projectName}/{projectName}.maiml')` → `parsePetriNetStructure(xmlContent)` → 失敗時 `MAIML_DOWNLOAD_FAILED` / `MAIML_PARSE_FAILED`
3. **メソッド検索**: `methods.find(m => m.methodId === methodId)` → なし時 `METHOD_NOT_FOUND`
4. **トランジション進捗取得（Promise.all 並列）**: 各トランジションについて下記を実行
   - MAIMLのプレース・アーク情報から `material_count`, `condition_count`, `result_count` を算出（stereotype別カウント）
   - `template_data/countActualByPlaceIds.sql` で実績数を取得
   - `transitionDataRepository.getLatestRecordByTransition()` で最新ステータスを取得
   - 取得したステータスと進捗率を `normalizeProgress()` で補正（後述）
5. **進捗率正規化（ `normalizeProgress()` ）**:
   - `status = 5`（COMPLETED）→ `progress` を強制的に `100` に設定
   - `status = 0`（INITIAL）→ `progress` を強制的に `0` に設定
   - DBレコードが存在しない場合 → `status = 0`、`progress = 0`、`last_updated = null` で返却
   - それ以外（1〜4, 6）→ `Math.max(0, Math.min(100, progress_rate))` で 0〜100 の範囲に制限
6. **全体進捗計算**: 各トランジションの `progress` 合計 / 総数 = `overall_percentage`

ミドルウェアチェーン:

```
router.get(
  '/projects/:projectName/methods/:methodId/tasks/:taskNo/transitions',
  authMiddleware,                    // 1. JWT認証
  checkPermission('050000', 0),      // 2. 権限チェック（実験作業者メニュー、読取専用）
  [
    validateParamString('projectName', 1, 255),
    validateParamString('methodId', 1, 255),
    validateParamInt('taskNo', 1),
  ],
  handleValidationErrors,            // 3. バリデーションエラーハンドリング
  progressController.getProgressData // 4. コントローラー
);
```

使用 SQL ファイル:

SQLファイル	説明
`backend/src/sql/queries/projects/findByName.sql`	プロジェクト名からプロジェクト情報取得
`backend/src/sql/queries/template_data/countActualByPlaceIds.sql`	プレースIDリストに対する実績データ数集計（template_typeごと）
`backend/src/repositories/transitionDataRepository.ts` - `getLatestRecordByTransition()`	トランジションの最新ステータス取得（transition_data テーブル）

---

4. タスク一覧取得（実験作業者用）API 🟢

エンドポイント: `GET /api/v1/progress/projects/:projectName/worker-tasks`

説明: プロジェクト内の全タスクをメソッド単位でグループ化して取得する

認証: 必須

権限: `experiment_worker`（機能CD: `050000`、読取専用）

優先度: ⭐⭐⭐ 高

実装状況: ✅ 実装済み

- コントローラー: `backend/src/controllers/progressController.ts` - `getWorkerTasks()`
- サービス: `backend/src/services/taskService.ts` - `TaskService.getTasksGroupedByMethod()`
- リポジトリ: `backend/src/repositories/projectTaskRepository.ts` - `findMethodsAndTasksByProject()`
- SQL: `backend/src/sql/queries/tasks/listMethodsAndTasksByProject.sql`
- ルート定義: `backend/src/routes/progressRoutes.ts` - `checkPermission('050000', 0)`

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`projectName`	string	○	プロジェクト名

クエリパラメータ（オプション）:

パラメータ	型	デフォルト	説明
`status`	string	-	タスクステータスフィルタ（`not_started` / `in_progress` / `completed` / `forced_terminated`）
`assignedTo`	string (UUID)	-	担当者IDフィルタ（内部では `assignUserId` として処理）
`sortBy`	string	`task_no`	ソート項目（`task_no` / `created_at` / `updated_at` / `status`）
`order`	string	`asc`	ソート順（`asc` / `desc`）

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "project_id": "550e8400-e29b-41d4-a716-446655440000",
    "project_name": "Sample Project",
    "methods": [
      {
```

```
        "method_id": "method1",
        "method_name": "method1",
        "experiment_order": 1,
        "tasks": [
          {
            "task_id": "task-detail-uuid-1",
            "task_no": 1,
            "task_status": "completed",
            "assign_user_id": "worker-uuid-1",
            "assign_user_name": "Worker Name 1",
            "progress_rate": 100,
            "created_at": "2026-01-21T10:00:00.000Z",
            "updated_at": "2026-01-21T14:00:00.000Z"
          },
          {
            "task_id": "task-detail-uuid-2",
            "task_no": 2,
            "task_status": "in_progress",
            "assign_user_id": "worker-uuid-2",
            "assign_user_name": "Worker Name 2",
            "progress_rate": 0,
            "created_at": "2026-01-21T14:00:00.000Z",
            "updated_at": "2026-01-21T15:30:00.000Z"
          }
        ]
      },
      {
        "method_id": "method2",
        "method_name": "method2",
        "experiment_order": 2,
        "tasks": []
      }
    ]
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```


レスポンス data フィールド詳細:

フィールド	型	説明
`project_id`	string (UUID)	プロジェクトID
`project_name`	string	プロジェクト名
`methods`	array	メソッドグループ一覧（experiment_order 昇順）

methods[] フィールド詳細:

フィールド	型	説明
`method_id`	string	メソッドID
`method_name`	string	メソッド名（現時点では method_id の値をそのまま返却）
`experiment_order`	number	実験順序
`tasks`	array	タスクサマリー一覧

tasks[] フィールド詳細（TaskSummary型）:

フィールド	型	説明
`task_id`	string	タスク詳細ID（`project_task_detail` の主キー）
`task_no`	number	タスク連番（`project_task_detail.task_no`）
`task_status`	string	タスクステータス文字列（`not_started` / `in_progress` / `completed` / `forced_terminated`）
`assign_user_id`	string | null	担当作業者ID
`assign_user_name`	string | null	担当作業者名
`progress_rate`	number | null	進捗率（0〜100）
`created_at`	string	作成日時（ISO 8601）
`updated_at`	string	更新日時（ISO 8601）

エラー時:

- 401: 認証エラー
- 403: 権限不足
- 404: プロジェクトが存在しない（`RESOURCE_NOT_FOUND`）
- 500: システムエラー

実装要件

コントローラー処理フロー:

```
// 1. projectName → projectId 解決（projects/findByName.sql）
const project = await pgp.oneOrNone(projectQuery, [projectName, organizationId]);
// → 404 RESOURCE_NOT_FOUND if not found

// 2. クエリパラメータをTaskFiltersに変換
const filters: TaskFilters = {
  methodId: req.query.methodId,
  status: req.query.status,
  assignUserId: req.query.assignedTo,  // assignedTo → assignUserId にマッピング
  sortBy: req.query.sortBy,
  order: req.query.order,
};

// 3. TaskService.getTasksGroupedByMethod() 呼び出し
const methodGroups = await taskService.getTasksGroupedByMethod(projectId, organizationId, filters);
```

TaskService 処理フロー:

1. `projectTaskRepository.findMethodsAndTasksByProject(projectId, organizationId, filters)` でレコード取得
2. `method_id` 単位でグループ化（Map使用）、データ整合性検証（不正な method_id 除外）
3. `experiment_order` 昇順でソート
4. `convertToTaskSummary()` で数値ステータス（0/1/3/4）を文字列（`not_started` / `in_progress` / `completed` / `forced_terminated`）に変換

ミドルウェアチェーン:

```
router.get(
  '/projects/:projectName/worker-tasks',
  authMiddleware,                    // 1. JWT認証
  checkPermission('050000', 0),      // 2. 権限チェック（実験作業者メニュー、読取専用）
  [validateParamString('projectName', 1, 255)],
  handleValidationErrors,            // 3. バリデーションエラーハンドリング
  progressController.getWorkerTasks  // 4. コントローラー
);
```

使用 SQL ファイル:

SQLファイル	説明
`backend/src/sql/queries/projects/findByName.sql`	プロジェクト名からプロジェクト情報取得
`backend/src/sql/queries/tasks/listMethodsAndTasksByProject.sql`	メソッド・タスク一覧取得（project_task と project_task_detail を JOIN）

タスクステータス数値 → 文字列 変換マップ:

DB値	文字列
0	`not_started`
1	`in_progress`
3	`completed`
4	`forced_terminated`

---

5. タスクステータス更新API 🟢

エンドポイント: `PATCH /api/v1/progress/projects/:projectName/methods/:methodId/tasks/:taskNo/status`

説明: プロジェクト名、メソッドID、タスク連番でタスクを特定してステータスを更新

認証: 必須

権限: `experiment_worker`（機能CD: `050000`、書込可）

優先度: ⭐⭐⭐ 高

実装状況: ✅ 実装済み

- コントローラー: `backend/src/controllers/progressController.ts` - `updateTaskStatus()`
- サービス: `backend/src/services/taskService.ts` - `updateTaskStatus()` 関数
- SQL: `backend/src/sql/queries/tasks/findByIdForUpdate.sql`（SELECT FOR UPDATE）
- SQL: `backend/src/sql/queries/tasks/updateStatus.sql`（UPDATE + 楽観的ロック）
- ルート定義: `backend/src/routes/progressRoutes.ts` - `checkPermission('050000', 1)`

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`projectName`	string	○	プロジェクト名
`methodId`	string	○	メソッドID（MAIMLファイル内のmethod id）
`taskNo`	Integer	○	タスク連番（1以上の整数）

リクエストボディ:

```
{
  "status": "not_started" | "in_progress" | "completed" | "forced_terminated",
  "currentUpdatedAt": "2026-02-18T10:00:00Z (ISO 8601形式, 楽観的ロック用, 必須)",
  "comment": "string (最大500文字, optional)"
}
```

リクエストボディのパラメータ詳細:

パラメータ	型	必須	説明	制約
`status`	string	○	更新後のステータス	`not_started` / `in_progress` / `completed` / `forced_terminated` のいずれか
`currentUpdatedAt`	string	○	楽観的ロック用タイムスタンプ	ISO 8601形式（必須）。DBのupdate_atと一致しない場合は409エラー
`comment`	string	×	コメント	最大500文字

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "task_no": 1,
    "project_name": "Sample Project",
    "method_id": "method1",
    "previous_status": null,
    "new_status": "completed",
    "updated_by": "user-uuid",
    "updated_by_name": "Worker Name",
    "updated_at": "2026-02-18T11:00:00Z",
    "comment": "実験完了しました"
  },
  "meta": {
    "timestamp": "2026-02-18T11:00:00Z",
    "requestId": "req_abc123"
  }
}
```

レスポンス data フィールド詳細:

フィールド	型	説明
`task_no`	number	タスク連番
`project_name`	string	プロジェクト名
`method_id`	string	メソッドID
`previous_status`	null	前のステータス（現在は常に `null`）
`new_status`	string	更新後のステータス（`not_started` / `in_progress` / `completed` / `forced_terminated`）
`updated_by`	string | null	更新したユーザーID（実験担当者ID）
`updated_by_name`	string | null	更新したユーザー名
`updated_at`	string	更新日時（ISO 8601）
`comment`	string | null	コメント（指定した場合）

エラー時:

- 400: バリデーションエラー（statusが無効、currentUpdatedAtが不正形式、等） / 無効なステータス遷移（`INVALID_STATUS_TRANSITION`）
- 401: 認証エラー
- 403: 権限不足
- 404: プロジェクト（`RESOURCE_NOT_FOUND`）またはタスク（`RESOURCE_NOT_FOUND`）が存在しない
- 409: プロジェクトステータスが編集不可（`PERMISSION_INSUFFICIENT`） / 楽観的ロック競合（`CONCURRENT_UPDATE_DETECTED`）
- 500: システムエラー（`SYSTEM_ERROR`）

ビジネスロジック

有効なステータス値:

文字列	DB値（数値）	説明
`not_started`	0	未着手
`in_progress`	1	実行中
`completed`	3	完了
`forced_terminated`	4	強制終了

ステータス遷移ルール（実装通り）:

```
const VALID_TRANSITIONS: Record<number, number[]> = {
  0: [1, 4],  // PENDING(not_started) → IN_PROGRESS(in_progress), FORCED_TERMINATED
  1: [3, 4],  // IN_PROGRESS → COMPLETED, FORCED_TERMINATED
  3: [],      // COMPLETED → 遷移不可（終端状態）
  4: []       // FORCED_TERMINATED → 遷移不可（終端状態）
};
```

プロジェクト編集可能ステータス（実装通り）:

- DB値 `1`（pending / タスク実行待ち）または `2`（in_progress / タスク実行中）のみ編集可能

- それ以外は 409 `PERMISSION_INSUFFICIENT` を返却

楽観的ロック制御:

- `currentUpdatedAt` はリクエストボディの**必須パラメータ**
- DBの `update_at` と比較し、不一致の場合は 409 `CONCURRENT_UPDATE_DETECTED` を返却
- SELECT FOR UPDATE でタスクを排他ロックした上でUPDATE実行

実装要件

コントローラー処理フロー:

```
// 1. projectName → projectId 解決（projects/findByName.sql）
const project = await pgp.oneOrNone(projectQuery, [projectName, organizationId]);
// → 404 RESOURCE_NOT_FOUND if not found

// 2. プロジェクト編集可能チェック（DB値 1 または 2 のみ編集可能）
const editableStatuses = [1, 2];
const projectStatus = Number(project.project_status);
if (!editableStatuses.includes(projectStatus)) {
  // → 409 PERMISSION_INSUFFICIENT
}

// 3. taskService.updateTaskStatus() 呼び出し（pg-promiseトランザクション）
const result = await updateTaskStatusService(
  organizationId, projectId, methodId, taskNo,
  userId, status, currentUpdatedAt, comment
);
```

taskService.updateTaskStatus() 処理フロー（pg-promiseトランザクション）:

1. `tasks/findByIdForUpdate.sql` で SELECT FOR UPDATE（organization_id, project_id, method_id, task_no でフィルタ）
2. タスクが存在しない場合は `RESOURCE_NOT_FOUND`
3. `validateTransition(currentStatus, newStatusNum)` でステータス遷移チェック → 失敗時 `INVALID_STATUS_TRANSITION`
4. `tasks/updateStatus.sql` で UPDATE（ `currentUpdatedAt` = DB の `update_at` が一致するレコードのみ）
5. UPDATE が 0件の場合（楽観的ロック失敗）→ `CONCURRENT_UPDATE_DETECTED`

ミドルウェアチェーン:

```
router.patch(
  '/projects/:projectName/methods/:methodId/tasks/:taskNo/status',
  authMiddleware,                    // 1. JWT認証
  checkPermission('050000', 1),      // 2. 権限チェック（実験作業者メニュー、書込可）
  [
    validateParamString('projectName', 1, 255),
    validateParamString('methodId', 1, 255),
    validateParamInt('taskNo', 1),
    body('status').notEmpty(),
```

```
    body('currentUpdatedAt').notEmpty(),
  ],
  handleValidationErrors,           // 3. バリデーションエラーハンドリング
  progressController.updateTaskStatus // 4. コントローラー
);
```

使用 SQL ファイル:

SQLファイル	説明
`backend/src/sql/queries/projects/findByName.sql`	プロジェクト名からプロジェクト情報取得
`backend/src/sql/queries/tasks/findByIdForUpdate.sql`	SELECT FOR UPDATE（タスク排他ロック）
`backend/src/sql/queries/tasks/updateStatus.sql`	タスクステータス更新（楽観的ロックチェック含む）

備考:

- `currentUpdatedAt` は**必須**パラメータ（省略不可）
- 有効なステータス値: `not_started` / `in_progress` / `completed` / `forced_terminated`（`started` は無効）
- COMPLETED（3）/ FORCED_TERMINATED（4）からは遷移不可（終端状態）
- `previous_status` はレスポンスに含まれるが、現在の実装では常に `null`
- **権限**: `experiment_worker`（機能CD: `050000`、書込可）のみ使用可能

---

6. トランジションステータス更新API 🟢

エンドポイント: `PATCH /api/v1/progress/projects/:projectName/methods/:methodId/transitions/:transitionId/status`

説明: トランジションのステータスを手動で更新します。進捗確認の一環としてトランジション単位の進捗を記録します。

認証: 必須

権限: `experiment_worker`（書込可）

優先度: ⭐⭐⭐ 高

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`projectName`	string	○	プロジェクト名
`methodId`	string	○	メソッドID（MAIMLファイル内のmethod id）
`transitionId`	string	○	トランジションID

リクエストボディ:

```
{
  "taskNo": 1,
```

```
  "status": "running",
  "progressRate": 50,
  "comment": "実験を開始しました。現在50%完了。"
}
```

リクエストボディのパラメータ詳細:

パラメータ	型	必須	説明	制約
taskNo	integer	○	タスク連番	1以上の整数
status	integer	○	トランジションステータス（数値）	0〜6の整数（0:初期, 1:準備中, 2:準備完了, 3:実行中, 4:一時停止, 5:完了, 6:エラー）
progressRate	integer	△	進捗率	0-100、statusが5=completedの場合は自動的に100
comment	string	×	コメント	最大2000文字

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "transitionId": "transition_001",
    "taskNo": 1,
    "methodId": "method_001",
    "projectName": "Sample Project",
    "status": "running",
    "progressRate": 50,
    "comment": "実験を開始しました。現在50%完了。",
    "updatedBy": "880e8400-e29b-41d4-a716-446655440003",
    "updatedAt": "2026-01-27T11:00:00Z"
  },
  "meta": {
    "timestamp": "2026-01-27T11:00:00Z",
    "requestId": "req_def456"
  }
}
```

エラー時 (400 Bad Request):

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Validation failed",
    "details": [
      {
        "field": "status",
        "message": "Invalid status value",
        "code": "INVALID_VALUE"
      }
    ]
```

```
  },
  "meta": {
    "timestamp": "2026-01-27T11:00:00Z",
    "requestId": "req_def456"
  }
}
```

エラー時 (403 Forbidden):

```
{
  "success": false,
  "error": {
    "code": "PERMISSION_INSUFFICIENT",
    "message": "Project is not in editable status"
  },
  "meta": {
    "timestamp": "2026-01-27T11:00:00Z",
    "requestId": "req_def456"
  }
}
```

エラー時 (404 Not Found):

```
{
  "success": false,
  "error": {
    "code": "RESOURCE_NOT_FOUND",
    "message": "Transition not found or does not belong to your organization"
  },
  "meta": {
    "timestamp": "2026-01-27T11:00:00Z",
    "requestId": "req_def456"
  }
}
```

ビジネスロジック

1. **プロジェクトステータス確認**
   - プロジェクトステータスが編集可能（draft/active/waiting_approval）であることを確認
2. **トランジション存在確認**
   - transitionIdが指定されたプロジェクト・メソッド・タスク連番に属することを確認
   - organization_idでマルチテナント分離
3. **ステータス更新**
   - transition_dataテーブルを更新
   - statusが"completed"の場合、progress_rateを自動的に100に設定
   - updated_byに現在のユーザーIDを設定
4. **イベントログ記録**

   - maiml_event_logテーブルに変更履歴を記録
   - 組織ID、プロジェクトIDはリクエストデータから取得
   - シーケンス番号は自動採番（organization_id、project_id単位で連番）
   - APIリクエストデータをJSON形式でlogカラムに格納

5. **タスク進捗の再計算（オプション）**
   - タスク内の全トランジションの進捗率から、タスク全体の進捗率を再計算

バリデーション

- [ ] projectName は文字列（パスパラメータ）
- [ ] methodId は文字列（パスパラメータ）
- [ ] transitionId は文字列（パスパラメータ）
- [ ] taskNo は1以上の整数（リクエストボディ）
- [ ] status は許可された値のいずれか
- [ ] progressRate は0-100の整数（指定時）
- [ ] comment は2000文字以内（指定時）

データベーステーブル

使用テーブル:

1. **transition_data**（主テーブル）
   - organization_id, project_id, method_id, task_no, transition_id
   - status, progress_rate, comment
   - updated_by, updated_at, created_at

2. **maiml_event_log**（イベントログ）
   - organization_id, project_id, sequence_no
   - log (JSONB)
   - created_at

3. **projects**（ステータス確認）
   - id, status, organization_id

4. **project_task**（存在確認）
   - project_id, task_no, organization_id

SQL例

```
-- トランザクション開始
BEGIN;
```

```
-- 1. プロジェクト情報取得
SELECT project_id, project_status FROM project
WHERE project_name = $1 AND organization_id = $2;

-- 2. プロジェクトステータス確認（編集可能）
-- draft, active, waiting_approval のいずれかであること

-- 3. タスク存在確認
SELECT task_no FROM project_task
WHERE project_id = $3
  AND task_no = $4
  AND organization_id = $2;

-- 4. トランジションステータス更新
UPDATE transition_data
SET
  status = $5,
  progress_rate = CASE
    WHEN $5 = 'completed' THEN 100
    ELSE $6
  END,
  comment = $7,
  updated_by = $8,
  updated_at = NOW()
WHERE organization_id = $2
  AND project_id = $3
  AND method_id = $9
  AND task_no = $4
  AND transition_id = $10
RETURNING *;

-- 5. イベントログ記録
INSERT INTO maiml_event_log (
  organization_id,
  project_id,
  sequence_no,
  log,
  created_at
) VALUES (
  $2,  -- organization_id
  $3,  -- project_id
  (
    SELECT COALESCE(MAX(sequence_no), 0) + 1
    FROM maiml_event_log
    WHERE organization_id = $2 AND project_id = $3
  ),  -- 自動採番されたsequence_no
  jsonb_build_object(
    'event_type', 'transition_status_update',
    'method_id', $9,
    'task_no', $4,
    'transition_id', $10,
    'status', $5,
    'progress_rate', CASE WHEN $5 = 'completed' THEN 100 ELSE $6 END,
    'comment', $7,
    'updated_by', $8,
    'timestamp', NOW()
  ),  -- APIデータをJSON形式で格納
  NOW()
);

-- 6. タスク進捗率の再計算（オプション）
UPDATE project_task
SET
```

```
  progress_rate = (
    SELECT AVG(progress_rate)
    FROM transition_data
    WHERE project_id = $3
      AND task_no = $4
      AND organization_id = $2
  ),
  updated_at = NOW()
WHERE project_id = $3
  AND task_no = $4
  AND organization_id = $2;

COMMIT;
```

パラメータ説明:

```
$1: project_name (string)
$2: organization_id (string)
$3: project_id (UUID) - プロジェクト名から取得
$4: task_no (integer) - リクエストボディ
$5: status (string) - リクエストボディ
$6: progress_rate (integer) - リクエストボディ
$7: comment (string) - リクエストボディ
$8: updated_by (UUID) - 現在のユーザーID
$9: method_id (string) - パスパラメータ
$10: transition_id (string) - パスパラメータ
```

セキュリティ

- [ ] organization_id フィルタ必須
- [ ] プロジェクトステータスが編集可能であることを確認
- [ ] SQLインジェクション対策（Parameterized Queries）
- [ ] パスパラメータの階層構造検証

パフォーマンス

- インデックス: transition_data (organization_id, project_id, method_id, task_no, transition_id) - 複合主キー
- インデックス: transition_data (organization_id, project_id, updated_at)
- トランザクション: 必須（整合性保証）

備考

- **旧エンドポイント**: この仕様は実験データ記録API（api_experiment_data.md）から移動
- **変更点**: taskNoをパスパラメータからリクエストボディパラメータに変更
- **権限**: experiment_worker（書込可）のみが使用可能
- **用途**: 進捗確認の一環としてトランジション単位の進捗を記録

実装状況

実装済み箇所:

- ✅ ルート定義: `backend/src/routes/progressRoutes.ts` (L120-L157)
  - `checkPermission('050000', 1)` による権限チェック実装済み
  - 機能CD: `050000` (実験作業者メニュー)、権限レベル: `1` (書込可)
- ✅ バリデーション: パスパラメータ、リクエストボディの検証実装済み
- ✅ コントローラー: `backend/src/controllers/progressController.ts` (L241-L324)
  - `updateTransitionStatus()` 関数実装済み
- ✅ プロジェクトステータス編集可能チェック実装済み

実装詳細:

- **権限チェックミドルウェア**: `backend/src/middleware/permissionCheckMiddleware.ts`
  - `checkPermission(functionId, permission)` 関数を使用
  - JWT認証後の `req.user.permissions` から権限情報を取得
  - 403エラーハンドリング実装済み

ミドルウェアチェーン:

```
router.patch(
  '/projects/:projectName/methods/:methodId/transitions/:transitionId/status',
  authMiddleware,                    // 1. JWT認証
  checkPermission('050000', 1),      // 2. 権限チェック（実装済み）
  [/* バリデーション */],             // 3. バリデーション
  handleValidationErrors,            // 4. エラーハンドリング
  progressController.updateTransitionStatus  // 5. コントローラー
);
```

参考: タスクステータス更新API（旧エンドポイント） 🔴

⚠️ **注意**: このエンドポイントは参考用です。新規実装では **API #5** の `PATCH /api/v1/progress/projects/:projectName/methods/:methodId/tasks/:taskNo/status` を使用してください。

エンドポイント: `PUT /api/v1/tasks/:taskId/status`

説明: タスクIDでタスクを特定してステータスを更新（旧仕様）

認証: 必須

権限: `experiment_admin`, `experiment_worker`, `system_admin`

優先度: ⭐⭐ 中

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`taskId`	UUID	○	タスクID

リクエストボディ:

```
{
  "status": "not_started" | "started" | "in_progress" | "completed" | "forced_terminated",
  "comment": "string (最大500文字, optional)"
}
```

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "task_id": "task-uuid",
    "task_name": "反応実験タスク",
    "previous_status": "in_progress",
    "new_status": "completed",
    "updated_by": "user-uuid",
    "updated_by_name": "User Name",
    "updated_at": "2026-01-21T16:00:00Z",
    "comment": "実験完了しました"
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: バリデーションエラー、または無効なステータス遷移
- 401: 認証エラー
- 403: 権限不足（担当者以外が更新しようとした）
- 404: タスクが存在しない
- 409: プロジェクトステータスが編集不可
- 500: システムエラー

ビジネスロジック

ステータス遷移ルール:

```
const validTransitions: Record<string, string[]> = {
  'not_started': ['started', 'forced_terminated'],
  'started': ['in_progress', 'forced_terminated'],
  'in_progress': ['completed', 'forced_terminated'],
  'completed': ['in_progress'], // 再開可能
  'forced_terminated': ['started'] // 再開可能
```

```
};

const canTransition = (from: string, to: string): boolean => {
  return validTransitions[from]?.includes(to) || false;
};
```

前提条件チェック:

```
// プロジェクトステータスが編集可能かチェック
const project = await db
  .selectFrom('project')
  .innerJoin('project_task', 'project.project_id', 'project_task.project_id')
  .where('project_task.task_id', '=', taskId)
  .where('project.organization_id', '=', organizationId)
  .select(['project.project_status'])
  .executeTakeFirstOrThrow();

const editableStatuses = ['draft', 'active'];
if (!editableStatuses.includes(project.project_status)) {
  throw new ConflictError(
    'プロジェクトステータスが編集可能な状態ではありません'
  );
}
```

実装要件

DBクエリ（トランザクション）:

```
await db.transaction().execute(async (trx) => {
  // 現在のタスク情報を取得
  const task = await trx
    .selectFrom('project_task')
    .where('task_id', '=', taskId)
    .where('organization_id', '=', organizationId)
    .select(['task_id', 'task_name', 'task_status', 'assign_user_id'])
    .executeTakeFirstOrThrow();

  // 権限チェック（Worker の場合は自分のタスクのみ）
  if (req.user.role === 'experiment_worker') {
    if (task.assign_user_id !== req.user.userId) {
      throw new ForbiddenError(
        '自分に割り当てられていないタスクは更新できません'
      );
    }
  }

  // ステータス遷移チェック
  if (!canTransition(task.task_status, status)) {
    throw new BadRequestError(
      `ステータスを ${task.task_status} から ${status} に変更できません`
    );
  }

  // ステータス更新
  await trx
    .updateTable('project_task')
    .set({
      task_status: status,
```

```
      update_at: new Date().toISOString(),
      update_user_id: req.user.userId
    })
    .where('task_id', '=', taskId)
    .where('organization_id', '=', organizationId)
    .execute();

  // コメントがある場合は記録（任意）
  if (comment) {
    await trx
      .insertInto('task_comment')
      .values({
        comment_id: uuidv4(),
        task_id: taskId,
        user_id: req.user.userId,
        comment: comment,
        organization_id: organizationId,
        create_at: new Date().toISOString()
      })
      .execute();
  }

  // 監査ログ記録
  logger.info('Task status updated', {
    taskId,
    previousStatus: task.task_status,
    newStatus: status,
    updatedBy: req.user.userId,
    organizationId
  });
});
```

バリデーション:

```
const validateStatusUpdate = (body: any) => {
  const validStatuses = [
    'not_started',
    'started',
    'in_progress',
    'completed',
    'forced_terminated'
  ];

  if (!body.status) {
    throw new BadRequestError('status is required');
  }

  if (!validStatuses.includes(body.status)) {
    throw new BadRequestError(
      `Invalid status. Must be one of: ${validStatuses.join(', ')}`
    );
  }

  if (body.comment && body.comment.length > 500) {
    throw new BadRequestError('comment must be 500 characters or less');
  }
};
```

---

7. 作業詳細一括取得API 🟢

エンドポイント: `GET /api/v1/progress/projects/:projectName/work-details`

説明: WorkProgressDetailDialog が必要とする projectInfo / fileInfo / workDetails（transitionProgress 含む）を一括返却する

認証: 必須

権限: `experiment_admin`（機能CD: `040000`、読取専用以上）

優先度: ⭐⭐⭐ 高

実装状況: ✅ 実装済み

- コントローラー: `backend/src/controllers/progressController.ts` - `getWorkDetails()`
- サービス: `backend/src/services/progressService.ts` - `ProgressService.getWorkDetails()`

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`projectName`	string	○	プロジェクト名

レスポンス

成功時 (200 OK):


```
{
  "success": true,
  "data": {
    "projectInfo": {
      "projectId": "550e8400-e29b-41d4-a716-446655440000",
      "projectName": "Sample Project",
      "projectDescription": "プロジェクトの説明",
      "owner": "Owner Name",
      "department": "研究部門",
      "status": "in_progress",
      "methods": ["method1", "method2"],
      "createdAt": "2026-01-21T10:00:00.000Z",
      "updatedAt": "2026-03-01T12:00:00.000Z"
    },
    "fileInfo": {
      "fileName": "Sample Project.maiml",
      "maimlFileKey": "550e8400-e29b-41d4-a716-446655440000/projects/Sample Project/Sample Project.maiml",
      "organizationId": "550e8400-e29b-41d4-a716-446655440000",
      "checkoutStatus": {
        "isCheckedOut": false,
        "checkoutBy": null
      }
    },
    "workDetails": [
      {
        "workId": "method1_1",
        "taskNo": 1,
        "methodName": "method1",
```

```
        "workStatus": "in_progress",
        "progressPercentage": 50,
        "stats": {
          "totalTransitions": 4,
          "completed": 2,
          "working": 1,
          "notStarted": 1,
          "completionRate": 50
        },
        "transitionProgress": [
          {
            "transitionId": "t1",
            "progressStatus": "完了",
            "assignedWorker": "Worker Name",
            "startDate": "2026-03-01T10:00:00.000Z",
            "endDate": "2026-03-01T14:00:00.000Z"
          },
          {
            "transitionId": "t2",
            "progressStatus": "作業中",
            "assignedWorker": "Worker Name 2",
            "startDate": "2026-03-02T09:00:00.000Z",
            "endDate": null
          },
          {
            "transitionId": "t3",
            "progressStatus": "未開始"
          }
        ]
      }
    ]
  },
  "meta": {
    "timestamp": "2026-03-11T00:00:00.000Z",
    "requestId": "req_abc123"
  }
}
```


projectInfo フィールド詳細:

フィールド	型	説明
`projectId`	string (UUID)	プロジェクトID
`projectName`	string	プロジェクト名
`projectDescription`	string | undefined	プロジェクト説明
`owner`	string	オーナー名（ユーザー名またはユーザーID）
`department`	string | undefined	部門名（または部門CD）
`status`	string	プロジェクトステータス（ `planning` / `pending` / `in_progress` / `review_waiting` / `completed` / `error` ）
`methods`	string[]	MAIMLファイルに定義されたメソッドID一覧
`createdAt`	string (ISO 8601)	作成日時
`updatedAt`	string (ISO 8601)	更新日時

workDetails[].stats フィールド詳細:

フィールド	型	説明
`totalTransitions`	number	MAIMLに定義されたトランジション総数
`completed`	number	完了数（status=5）
`working`	number	作業中数（status=1〜4）
`notStarted`	number	未開始数
`completionRate`	number	完了率（0〜100の整数）

workDetails[].transitionProgress[].progressStatus 値:

値	DBのstatus値	説明
未開始	DBに未登録 or 0	初期状態
準備中	1	準備中
作業中	2 または 4	作業中・再開
休止中	3	一時停止
完了	5	完了
異常中	6	エラー状態

エラー時:

- 401: 認証エラー
- 403: 権限不足
- 404: プロジェクトが存在しない（ `TASK_NOT_FOUND` ）
- 500: MAIMLダウンロード失敗 / MAIMLパース失敗 / DBエラー

ビジネスロジック

1. **プロジェクト基本情報取得**
   - `projects/findWithOwnerAndDept.sql` を使用し、プロジェクト名と組織IDでプロジェクト情報・オーナー名・部門名を取得
2. **MAIML ダウンロード & パース**
   - S3/Swift から `{organizationId}/projects/{projectName}/{projectName}.maiml` をダウンロード
   - `parsePetriNetStructure()` でメソッド一覧とトランジション一覧を抽出
3. **タスク一覧取得**
   - `project_tasks/findByProjectId.sql` でプロジェクト内の全タスクを取得
4. **トランジション進捗一括取得（Promise.all 並列処理）**
   - タスクごとに `project_tasks/findTaskDetailsByMethodId.sql` で task_no 一覧を取得

- 各 task_no の `transition_data/findByTask.sql` でトランジション進捗を取得
- MAIML に定義されているが DB に未登録のトランジションは「未開始」として返す

5. **統計情報算出**
   - `completedCount` : status=5 のトランジション数
   - `workingCount` : status=1〜4 のトランジション数
   - `completionRate` : completedCount / totalTransitions × 100（整数）
   - `workStatus` : 全完了なら `completed` 、1件でも作業中なら `in_progress` 、それ以外は `pending`

ミドルウェアチェーン

```
router.get(
  '/projects/:projectName/work-details',
  authMiddleware,                    // 1. JWT認証
  checkPermission('040000', 0),      // 2. 権限チェック（実験管理者メニュー、読取専用以上）
  getWorkDetails                     // 3. コントローラー
);
```

使用SQLファイル

SQLファイル	用途
`projects/findWithOwnerAndDept.sql`	プロジェクト基本情報（オーナー・部門含む）取得
`project_tasks/findByProjectId.sql`	プロジェクト内タスク一覧取得
`project_tasks/findTaskDetailsByMethodId.sql`	メソッドIDに紐づく task_no 一覧取得
`transition_data/findByTask.sql`	タスク（task_no）のトランジション進捗一覧取得

備考

- **用途**: WorkProgressDetailDialog で使用（進捗確認画面）
- **workId の形式**: `{methodId}_{experimentOrder}` （例: `method1_1` ）
- **methodName フィールド**: 現時点では `method_id` の値をそのまま返却（MAIML のメソッド名表示名は未対応）
- MAIMLに存在するがDBに未登録のトランジションは `progressStatus: '未開始'` として含める
- プロジェクト内の全タスクを対象とする（担当者フィルタなし）

📚 関連ドキュメント

規約

- API設計ポリシー

- バックエンド規約
- セキュリティ規約

設計書

- API一覧（統合版）
- API実装方針
- データベース構造
- プロジェクト管理API
- Maiml編集API

UI仕様書

- 進捗確認画面

📝 変更履歴

バージョン	日付	変更内容	担当者
2.0.0	2026-03-12	トランジションステータス更新API（API #6）仕様書修正（ソースコード実装に合わせて更新）	Cline
		- API #6 見出しを 🔴（未実装）→ 🟢（実装済み）に変更	
		- ビジネスロジック #1 プロジェクト編集可能ステータスを実装に合わせて修正（`project_status = 1 or 2` の数値チェック）	
		- 実装状況セクションを詳細化（progressController.ts の `updateTransitionStatus()` 関数の実装情報を追記）	
		- コントローラーから `organizationId` / `updatedBy` を `req.user` から取得する実装詳細を追記	
		- `ProgressService.updateTransitionStatus()` 経由でトランジション更新・イベントログ記録する実装を追記	
1.9.0	2026-03-12	進捗データ取得API（API #2）仕様書修正（ソースコード実装に合わせて更新）	Cline
		- transitions[].progress フィールド説明を修正（status=5時に100、status=0時に0へ補正される旨を追記）	
		- overall_progress.in_progress フィールド説明を修正（ステータス名称を明示: PREPARING/READY/IN_PROGRESS/PAUSED/ERROR）	
		- ProgressService処理フローにステップ5「進捗率正規化（normalizeProgress()）」を追加	
		- normalizeProgress(): status=5→100固定、status=0→0固定、DBレコードなし→status=0/progress=0/last_updated=null、それ以外→0〜100クランプ	
1.8.0	2026-03-12	タスク関連API #3/#4/#5 仕様書修正（ソースコード実装に合わせて更新）	Cline
		- 実装状況サマリー: 3→6実装済み、3→0未実装、完了率100%に更新	
		- API一覧テーブル: #3/#4/#5を 🔴未実装→🟢実装済みに変更	
		- API #3（進捗データ取得）: 🔴→🟢、実装状況セクション追加	
		- API #3 レスポンス: statusを文字列→数値（0〜6）に修正、フィールド詳細表追加	

		- API #3 レスポンス: method_nameはmethod_idを返却、task_noは整数型に修正	
		- API #3 実装要件: DBクエリ（仮実装）→ProgressService処理フロー・ミドルウェアチェーン・SQLファイル一覧に差し替え	
		- API #4（タスク一覧取得）: 🔴→🟢、実装状況セクション追加	
		- API #4 レスポンス: tasks[]をTaskSummary型（task_id/task_status/assign_user_name/progress_rate等）に修正	
		- API #4 リクエスト: パラメータ表を修正（パスパラメータ/クエリパラメータに分離）	
		- API #4 実装要件: TaskService.getTasksGroupedByMethod()処理フロー・SQLファイル一覧に差し替え	
		- API #5（タスクステータス更新）: 🔴→🟢、実装状況セクション追加	
		- API #5 リクエスト: currentUpdatedAtを必須パラメータに変更（optional→必須）	
		- API #5 ステータス値: startedを削除、有効値を4種類（not_started/in_progress/completed/forced_terminated）に修正	
		- API #5 ステータス遷移ルール: 実装通りの遷移ルール（VALID_TRANSITIONS数値マップ）に差し替え	
		- API #5 プロジェクト編集可能チェック: editableStatuses=[1,2]（数値）に修正	
		- API #5 実装要件: pg-promiseトランザクション処理フロー・SQLファイル一覧に差し替え	
		- API #5 previous_status: 現在の実装では常にnullと明記	
1.7.0	2026-03-11	タスク詳細取得API 仕様書修正（実装に合わせて更新）	Cline
		- API #1: エンドポイント・パラメータ・レスポンス・権限を実際の実装に合わせて修正	
		- パラメータ: projectName / methodId / taskNo（パスパラメータ3種に変更）	
		- クエリパラメータ（include_records, include_issues）削除	
		- レスポンス: TaskDetailResponse 型に合わせて更新（transitionProgress[] + stats{}）	
		- 権限: checkPermission('040000', 0) = experiment_admin 読取専用	
		- 実装要件: SQLファイルベース（findDetailByTaskId.sql / findTransitionsByTaskId.sql）に差し替え	
		- 実装状況セクション追加（progressRoutes.ts / projectController.ts / projectTaskDetailRepository.ts）	
1.6.0	2026-03-11	作業詳細一括取得API追加（7エンドポイント）	Cline
		- API #7: GET /api/v1/progress/projects/:projectName/work-details 新規追加（実装済み）	
		- WorkProgressDetailDialog 向けの projectInfo / fileInfo / workDetails 一括返却	
		- 権限: experiment_admin（機能CD: 040000、読取専用以上）	
		- 実装済み: progressController.ts `getWorkDetails()` 、progressService.ts `ProgressService.getWorkDetails()`	
		- 総API数: 6→7（+1）、実装済み: 2→3（+1）	
		- 完了率: 33.3%→42.9%	
1.5.0	2026-03-09	タスク詳細取得API エンドポイント変更・実装済みに更新	Cline
		- API #2: エンドポイントを `GET /api/v1/tasks/:taskId` → `GET /api/v1/progress/projects/:projectName/methods/:methodId/tasks/:taskNo/details` に変更	
		- 実装済み（ `backend/src/controllers/projectController.ts` `getTaskDetail()` 、 `backend/src/routes/progressRoutes.ts` ）	
		- パラメータ: taskId(UUID) → projectName/methodId/taskNo の組み合わせに変更	
		- `server.ts` から旧 `tasksRoutes` の登録を削除	

		- 実装状況サマリー: 1→2実装済み、5→4未実装	
		- 完了率: 16.7%→33.3%	
1.4.2	2026-02-26	進捗データ取得API レスポンス仕様修正	Cline
		- API #3: 各トランジションに材料・条件・結果の定義数と実績数を追加	
		- `material_count`: MAIMLファイルから取得したトランジションに紐づく材料の定義数	
		- `material_actual_count`: DBから取得した材料の実績数	
		- `condition_count`: MAIMLファイルから取得したトランジションに紐づく条件の定義数	
		- `condition_actual_count`: DBから取得した条件の実績数	
		- `result_count`: MAIMLファイルから取得したトランジションに紐づく結果の定義数	
		- `result_actual_count`: DBから取得した結果の実績数	
		- 実装要件にMAIMLパーサーによる定義数取得処理を追記	
		- 実装要件にexperiment_record_detailテーブルからの実績カウントクエリを追記	
1.4.0	2026-02-18	トランジションステータス更新API追加（6エンドポイント）	Cline
		- API #6: PATCH /api/v1/progress/projects/:projectName/methods/:methodId/transitions/:transitionId/status	
		- 実験データ記録APIから進捗確認APIに移動	
		- taskNoをリクエストボディパラメータ化	
		- 権限: experiment_worker（書込可）	
		- トランジション単位の進捗更新	
		- イベントログ記録とタスク進捗率再計算	
1.3.0	2026-02-18	タスク一覧取得（実験作業者用）API仕様変更（5エンドポイント）	Cline
		- API #4: GET /api/v1/progress/projects/:projectName/worker-tasks（エンドポイント変更）	
		- タスク一覧取得（プロジェクト単位）からタスク一覧取得（実験作業者用）に変更	
		- プロジェクト名のみで特定（methodIdパラメータ削除）	
		- メソッドごとにグループ化したレスポンス	
		- 総API数: 6→5（-1: 重複削除）	
1.2.0	2026-02-18	タスク一覧取得（実験作業者用）API追加（6エンドポイント）	Cline
		- API #6: GET /api/v1/progress/projects/:projectName/methods/:methodId/worker-tasks	
		- 実験作業者が自分に割り当てられたタスクの一覧を取得	
		- 権限: experiment_worker（読取専用）	
1.1.0	2026-02-18	タスクステータス更新API追加（5エンドポイント）	Cline
		- API #5: PATCH /api/v1/progress/projects/:projectName/methods/:methodId/tasks/:taskNo/status	
		- プロジェクト名+メソッドID+タスク連番でタスク特定	
		- 楽観的ロック（currentUpdatedAt）実装（オプション）	
		- 旧エンドポイント（PUT /api/v1/tasks/:taskId/status）はAPI #6として参考情報化	

1.0.0	2026-01-21	新規作成	Cline

作成者: Cline
作成日: 2026-01-21 16:45
最終更新: 2026-02-18 14:00